ECE276B Project 3 Report: Infinite-Horizon Stochastic Optimal Control

Stephen Huang
Department of Electrical and Computer Engineering
University of California, San Diego
La Jolla, U.S.A.
sth004@ucsd.edu

***Abstract*—In this project, we study safe trajectory tracking for a stochastic differential-drive robot. The robot must track a periodic lemniscate reference trajectory while avoiding circular obstacles under Gaussian motion noise and bounded control inputs. We compare two approaches: receding-horizon certainty equivalent control (CEC), which solves a deterministic nonlinear program online, and generalized policy iteration (GPI), which solves a discretized stochastic optimal control problem. We evaluate both methods in terms of tracking performance, collision avoidance, and computational cost.**

I. Introduction

Trajectory tracking is important in robotics because a robot must follow a desired path while accounting for dynamics, uncertainty, and safety constraints. In this project, we consider a differential-drive robot that tracks a figure-eight reference trajectory in a bounded planar environment with circular obstacles. The robot is affected by Gaussian motion noise, and its control input is constrained by linear and angular velocities.

The objective is to design a control policy that keeps the robot close to the reference position and orientation while avoiding collisions. This is formulated as a discounted infinite-horizon stochastic optimal control problem over the tracking error state. The stage cost penalizes position error, orientation error, and control effort, while the safety constraint requires the robot body to remain in free space.

We implement and compare two approaches for this problem. The first method is receding-horizon certainty equivalent control, which ignores the motion noise during optimization and repeatedly solves a deterministic finite-horizon nonlinear program. The second method is generalized policy iteration, which discretizes the error state and control spaces to compute a stochastic feedback policy. These methods demonstrate the tradeoff between online optimization and offline dynamic programming for safe stochastic trajectory tracking.

II. Problem Statement

We formulate safe trajectory tracking as a discounted infinite-horizon stochastic optimal control problem.

A. State and Control Space

Consider a differential-drive robot with state

$$x_t = \begin{bmatrix} p_t^\top & \theta_t \end{bmatrix}^\top, \qquad p_t \in \mathbb{R}^2, \quad \theta_t \in [-\pi, \pi), \tag{1}$$

where p_t denotes position and θ_t denotes orientation.

The control input is defined as

$$u_t = \begin{bmatrix} v_t & \omega_t \end{bmatrix}^\top \in \mathcal{U} = [0.1, 1] \times [-1, 1], \tag{2}$$

where v_t and ω_t are the linear and angular velocities.

B. Robot Dynamics and Environment

The robot evolves according to the stochastic discrete-time dynamics

$$x_{t+1} = \begin{bmatrix} p_{t+1} \\ \theta_{t+1} \end{bmatrix} = f(x_t, u_t, w_t) = \begin{bmatrix} p_t \\ \theta_t \end{bmatrix} + A \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} + w_t, \tag{3}$$

$$A = \begin{bmatrix} \Delta \operatorname{sinc}\left(\frac{\omega_t \Delta}{2}\right) \cos\left(\theta_t + \frac{\omega_t \Delta}{2}\right) & 0 \\ \Delta \operatorname{sinc}\left(\frac{\omega_t \Delta}{2}\right) \sin\left(\theta_t + \frac{\omega_t \Delta}{2}\right) & 0 \\ 0 & \Delta \end{bmatrix}$$

where

$$w_t \sim \mathcal{N}(0, \operatorname{diag}(\sigma)^2), \qquad \sigma = [0.04,\ 0.04,\ 0.004]^\top, \tag{4}$$

models independent Gaussian motion noise.

The robot operates in a bounded space

$$[-3, 3]^2 \subset \mathbb{R}^2 \tag{5}$$

containing four circular obstacles:

$$C_1 = (2.35, 0.95),\ C_2 = (-2.35, -0.95), \tag{6}$$

$$C_3 = (1, 0),\ C_4 = (-1, 0),$$

each with radius

$$R_o = 0.5. \tag{7}$$

The robot body is modeled as a Euclidean ball

$$\mathcal{B}_{0.3}(p_t) = \{z \in \mathbb{R}^2 \mid \|z - p_t\| \le 0.3\}, \tag{8}$$

centered at the robot position p_t.

The free space is therefore defined as

$$\mathcal{F} = [-3, 3]^2 \setminus \bigcup_{q \in (C_1 \cup C_2 \cup C_3 \cup C_4)} \mathcal{B}_{0.3}(q), \tag{9}$$

which represents all feasible robot positions that remain inside the bounded space while avoiding obstacle collisions.

C. Tracking Error

The objective is to track a periodic reference trajectory with a desired position and orientation

$$r_t \in \mathbb{R}^2, \quad \alpha_t \in [-\pi, \pi). \tag{10}$$

We define the tracking error state as

$$e_t = \begin{bmatrix} \tilde{p}_t^\top & \tilde{\theta}_t \end{bmatrix}^\top, \tag{11}$$

where

$$\tilde{p}_t = p_t - r_t, \qquad \tilde{\theta}_t = \theta_t - \alpha_t. \tag{12}$$

D. Stochastic Optimal Control Objective

We want to find a feedback policy

$$\pi(t, e_t) \in \mathcal{U} \tag{13}$$

that minimizes the discounted infinite-horizon expected cost

$$V^*(\tau, e) = \min_\pi \mathbb{E}\left[\sum_{t=\tau}^{\infty} \gamma^{t-\tau}\left(\tilde{p}_t^\top Q \tilde{p}_t + q(1-\cos(\tilde{\theta}_t))^2 + u_t^\top R u_t\right) \middle| e_\tau = e\right] \tag{14}$$

The optimization is performed subject to stochastic dynamics, control limits, and safety constraints:

$$u_t = \pi(t, e_t) \in \mathcal{U}, \qquad \tilde{p}_t + r_t \in \mathcal{F}. \tag{15}$$

Here, Q penalizes position tracking error, q penalizes orientation tracking error, and R penalizes excessive control effort. The safety constraint encourages the robot to remain in free space while tracking the reference trajectory under noise.

III. Technical Approach

We discuss the technical approach to CEC and GPI respectively.

A. Certainty Equivalent Control (CEC)

We implement receding-horizon certainty equivalent control (CEC) to approximately solve the discounted infinite-horizon stochastic optimal control problem. Instead of explicitly optimizing over motion uncertainty, CEC assumes the disturbance is fixed at its expected value,

$$w_t = 0, \tag{16}$$

which converts the stochastic problem into a deterministic finite-horizon optimization problem.

At each control step, only the first control input is applied to the robot. The optimization is then repeated online using the updated robot state and reference trajectory. This repeated replanning allows the controller to react to accumulated tracking error caused by stochastic motion noise.

1) Controller Configuration: The controller parameters were selected empirically to balance tracking performance, control smoothness, and obstacle avoidance.

The planning horizon was set to

$$T = 59 \tag{17}$$

steps to allow the optimizer to anticipate future obstacle interactions while maintaining reasonable solve times.

The stage cost weights were chosen as

$$Q = \mathrm{diag}(10, 10), \qquad q = 2, \qquad R = \mathrm{diag}(5, 0.5), \tag{18}$$

where Q penalizes position tracking error, q penalizes orientation error, and R penalizes control effort. The larger penalty on linear velocity discourages aggressive forward motion near obstacles.

To encourage convergence near the end of the prediction horizon, terminal costs were added:

$$Q_T = \mathrm{diag}(20, 20), \qquad q_T = 4. \tag{19}$$

The discount factor was selected as

$$\gamma = 0.84 \tag{20}$$

to prioritize near-term tracking performance while still considering future states.

2) Nonlinear Program Formulation: The optimization variables consist of the predicted control sequence

$$U = [u_\tau^\top, \ldots, u_{\tau+T-1}^\top]^\top \tag{21}$$

and the corresponding predicted error trajectory

$$E = [e_\tau^\top, \ldots, e_{\tau+T}^\top]^\top. \tag{22}$$

The optimization objective minimizes discounted tracking error and control effort:

$$J(U, E) = \sum_{k=0}^{T-1} \gamma^k \left(\tilde{p}_k^\top Q \tilde{p}_k + q(1-\cos(\tilde{\theta}_k))^2 + u_k^\top R u_k\right) + q_T(e_T), \tag{23}$$

where

$$q_T(e_T) = \tilde{p}_T^\top Q_T \tilde{p}_T + q_T(1-\cos(\tilde{\theta}_T))^2. \tag{24}$$

The optimization is subject to three constraints:

Initial condition constraint

$$e_0 = e_{\text{current}}. \tag{25}$$

Dynamics constraints

$$e_{k+1} = g(k, e_k, u_k, 0), \tag{26}$$

which enforce deterministic propagation of the error-state trajectory.

Control constraints

$$0.1 \le v_k \le 1, \qquad -1 \le \omega_k \le 1. \tag{27}$$

The resulting NLP is solved online using IPOPT through CasADi.

3) Collision Avoidance Through Obstacle Inflation: Collision avoidance is enforced using hard inequality constraints instead of collision penalties.

For every predicted state, the robot position is constrained to remain inside the bounded space:

$$-3 \le x_k \le 3, \qquad -3 \le y_k \le 3, \tag{28}$$

and maintain a minimum distance from each obstacle:

$$\|p_k - c_i\|^2 \ge (R_b + R_o + m)^2. \tag{29}$$

Here, $R_b = 0.3$ is the robot radius, $R_o = 0.5$ is the obstacle radius, and m is an additional safety margin.

Initially, a nearly zero margin was sufficient for deterministic simulations. However, under Gaussian motion noise, trajectories would frequently collide into obstacles. This is especially true because of the thin gap between two obstacles (C_1, C_3) and (C_2, C_4). Therefore, we increase the obstacle radius through an additional noise margin:

$$m = m_{\text{base}} + m_{\text{noise}}, \tag{30}$$

where

$$m_{\text{base}} = 10^{-5}, \qquad m_{\text{noise}} = 0.15. \tag{31}$$

This biases the optimizer toward safer paths and reduces collision frequency.

4) Warm-Start Initialization: To improve computational efficiency, we warm-start the optimizer using the previous optimal control sequence.

After solving at time t, the solution is shifted forward:

$$U^{(0)} = [u_1^*, u_2^*, \ldots, u_T^*, u_T^*], \tag{32}$$

and used as the initialization for the next optimization.

If no previous solution exists, the initial control sequence is generated from the reference trajectory velocities.

Warm starting significantly reduced solve time and improved optimization stability, especially when obstacle constraints became active.

Algorithm 1 Warm-Start Control Initialization

function INITIALIZECONTROL($U_{\text{prev}}, \{r_k, \alpha_k\}_{k=0}^{N}$)
  if U_{prev} exists **then**
    Shift previous U_{prev} forward by one step
    Repeat the final control at the last horizon step
    return shifted control sequence
  else
    for $k = 0$ to $N-1$ **do**
      Compute velocity from $\|r_{k+1} - r_k\|/\Delta$
      Compute angular velocity from $\text{wrap}(\alpha_{k+1} - \alpha_k)/\Delta$
      Clip to control bounds
    end for
    return feedforward control sequence
  end if
end function

5) CEC Implementation: We now provide a high-level overview of our algorithm.

Algorithm 2 CEC Controller Implementation

function CECCONTROLLER(x_t, t, U_{prev})
  Get local segment $\{r_k, \alpha_k\}_{k=t}^{t+N}$
  Compute current tracking error e_t
  $U^{(0)} \leftarrow$ INITIALIZECONTROL($U_{\text{prev}}, \{r_k, \alpha_k\}_{k=t}^{t+N}$)
  Construct nonlinear program with decision variables E and U
  Add initial condition constraint
  for $k = 0$ to $N-1$ **do**
    Add deterministic error-state dynamics constraint
    Add control bounds
    Add space constraints
    Add inflated-obstacle constraints
    Add discounted stage cost
  end for
  Add terminal cost
  Solve NLP with IPOPT
  Store optimized control sequence for warm-starting
  return first control input
end function

B. Generalized Policy Iteration (GPI)

We solve the stochastic tracking problem directly by discretizing the error-state and control spaces and formulating it as a finite, time-periodic Markov Decision Process (MDP). The policy is computed offline using generalized policy iteration, then applied online through a table lookup.

1) Grid Value Function: The GPI controller uses a grid-based value function representation. Let the discretized error-state grids be

$$\mathcal{E}_x = \{e_x^1, \ldots, e_x^{n_x}\}, \qquad \mathcal{E}_y = \{e_y^1, \ldots, e_y^{n_y}\}, \tag{33}$$

$$\mathcal{E}_\theta = \{e_\theta^1, \ldots, e_\theta^{n_\theta}\}.$$

Since the reference trajectory is periodic with period T, the value function is represented as a four-dimensional table

$$V \in \mathbb{R}^{T \times n_x \times n_y \times n_\theta}, \tag{34}$$

where each entry stores the cost-to-go for one discrete time index and one discrete error state:

$$V(t, e_x^i, e_y^j, e_\theta^k). \tag{35}$$

For faster vectorized computation, the value table is also reshaped into a flattened view of size

$$T \times (n_x n_y n_\theta), \tag{36}$$

so that transition updates can be applied efficiently.

The grid value function supports three main operations: direct lookup, in-place assignment of updated Bellman values, and copying for reuse or caching.

2) GPI Setup: The GPI controller is constructed in a separate setup file. We first create the discretized state and control spaces, then initialize a grid value function and constructs the corresponding `GpiConfig` object. The values we use are

$$n_\theta = 10, \quad n_v = 9, \quad n_\omega = 8, \quad \gamma = 0.84, \quad N_{\text{eval}} = 20, \tag{37}$$
$$n_{\text{fine}} = 22, \quad m_{coll} = 0.0001$$

with stage-cost weights

$$Q = \text{diag}(100, 100), \qquad q = 10, \qquad R = \text{diag}(3, 0.5). \tag{38}$$

The setup first attempts to load a cached policy from disk, and if no cache exists, it computes the policy offline and stores it for future runs.

Algorithm 3 GPI Setup and Construction

```
function MAKEGPI(obstacles, parameters)
    Build dense/coarse error grids Ex, Ey
    Build uniform orientation grid Eθ
    Build uniform control grids V and W
    Initialize grid value function V(t, e_x, e_y, eθ)
    Construct GpiConfig using trajectory, obstacles, weights, and grids
    Create GPI controller from configuration
    if cached policy exists and recomputation is not requested then
        Load cached policy and value table
    else
        Compute offline GPI policy
    end if
    return GPI controller
end function
```

3) State and Control Discretization: The tracking-error state is

$$e_t = \begin{bmatrix} \tilde{x}_t & \tilde{y}_t & \tilde{\theta}_t \end{bmatrix}^\top. \tag{39}$$

The full state is given by the pair (t, e), where t is the periodic reference index and

$$e \in \mathcal{E}_x \times \mathcal{E}y \times \mathcal{E}_\theta. \tag{40}$$

The control input is discretized as

$$u = \begin{bmatrix} v & \omega \end{bmatrix}^\top, \qquad v \in \mathcal{V}, \qquad \omega \in \mathcal{W}. \tag{41}$$

Thus, each discrete action corresponds to one pair (v, ω) from the control grids.

This gives us a graph construction with

$$|\mathcal{X}| = T n_x n_y n_\theta \tag{42}$$

states and

$$|\mathcal{U}| = n_v n_\omega \tag{43}$$

controls per state.

4) Stochastic Transition Model: For each time index t, error state e, and control u, we first compute the next state using the error dynamics without noise:

$$\bar{e}_{t+1} = g(t, e, u, 0). \tag{44}$$

Because the system is stochastic, the true next state is distributed around this mean according to the Gaussian motion-noise model

$$w_t \sim \mathcal{N}(0, \text{diag}(\sigma)^2). \tag{45}$$

$$\frac{\exp(\log \tilde{p}_j)}{\sum_{k=1}^{8} \exp(\log \tilde{p}_k)}, \qquad \sum_{j=1}^{8} p_j = 1. \tag{46}$$

This gives a sparse stochastic transition model:

$$P(e' \mid t, e, u), \tag{47}$$

where only eight successor states have nonzero probability for each state-action pair. All transition corner indices and probabilities are precomputed offline and stored for optimized Bellman updates.

5) Stage Cost and Collision Penalty: For each discrete error state

$$e = \begin{bmatrix} \tilde{x} & \tilde{y} & \tilde{\theta} \end{bmatrix}^\top \tag{48}$$

and discrete control

$$u = \begin{bmatrix} v & \omega \end{bmatrix}^\top, \tag{49}$$

the base stage cost is defined as

$$\ell(e, u) = \tilde{p}^\top Q \tilde{p} + q(1 - \cos\tilde{\theta})^2 + u^\top R u. \tag{50}$$

In implementation, this cost is precomputed for every discrete state-action pair and stored as a table

$$\ell(e, u) \in \mathbb{R}^{n_x n_y n_\theta \times n_u}. \tag{51}$$

The GPI controller uses the following values:

$$Q = \text{diag}(100, 100), \qquad q = 10, \qquad R = \text{diag}(3, 0.5). \tag{52}$$

To prevent obstacle collision, each obstacle is inflated with

$$R_{\text{safe},i} = R_i + R_b + m, \tag{53}$$

where R_i is the obstacle radius, $R_b = 0.3$ is the robot radius, and m is the additional collision margin chosen in the GPI setup.

Instead of deleting collision states, we assign them a very large penalty:

$$c(t, e) = 10^{10}. \tag{54}$$

This penalty is applied whenever the reconstructed robot position lies inside an inflated obstacle or leaves the bounded space.

Therefore, the effective stage cost used by policy iteration is

$$\ell'(t, e, u) = \ell(e, u) + c(t, e). \tag{55}$$

6) Policy Evaluation: Given the current policy π, policy evaluation updates the grid value function using the Bellman expectation equation

$$V^{\pi}(t,e) = \ell(t,e,\pi(t,e)) + \gamma \sum_{e'} P(e' \mid t,e,\pi(t,e)) V^{\pi}(t+1,e'), \tag{56}$$

where

$$t+1 = (t+1) \bmod T \tag{57}$$

because the reference trajectory is periodic.

We precompute the transitions using eight neighboring states for each discrete state-action pair. The expected next value is therefore evaluated as

$$\sum_{j=1}^{8} w_j(t,e,u)\, V(t+1,e'_j), \tag{58}$$

where e'_j are the stored next corner states and w_j is the normalized transition weights.

We attempt to run policy evaluation until full convergence, but implement a fixed cap to improve performance:

$$N_{\text{eval}} = 20. \tag{59}$$

We also initialize the value function as

$$V(t,e) = 0 \tag{60}$$

before the GPI iterations begin.

7) Policy Improvement: After policy evaluation, the policy is updated by minimizing the discrete action-value function:

$$\pi'(t,e) = \arg\min_{u \in \mathcal{U}} \left[\ell(t,e,u) + \gamma \sum_{e'} P(e' \mid t,e,u) V(t+1,e') \right]. \tag{61}$$

Since the transition model and stage costs are precomputed, the action values for all controls can be evaluated efficiently.

We continue to iterate until the next policy matches the previous policy.

Algorithm 4 Generalized Policy Iteration

function COMPUTEPOLICY
  Precompute transition corner indices and transition probabilities
  Precompute stage costs $\ell(e,u)$
  Precompute collision penalties $c(t,e)$
  Initialize value table $V(t,e) \leftarrow 0$ and policy table $\pi(t,e) \leftarrow 0$
  for iteration $= 1$ to N_{GPI} **do**
    for $j = 1$ to N_{eval} **do**
      for $t = T-1$ down to 0 **do**
        Evaluate $V(t,e)$ under the current policy
      end for
    end for
    for $t = 0$ to $T-1$ **do**
      for each discrete state e **do**
        Compute action values for all discrete controls
        Update $\pi(t,e)$ using the minimizing action
      end for
    end for
    if policy unchanged **then**
      break
    end if
  end for
  Save policy and value function to disk
end function

8) Online Execution: After the offline GPI computation is complete, control is executed online through a policy lookup. Given the current robot state x_t and reference state x_t^{ref}, the controller computes the tracking error

$$e_t = x_t - x_{t,\text{ref}}, \tag{62}$$

then wraps the orientation error into $[-\pi, \pi)$:

$$\tilde{\theta}_t = \operatorname{atan2}(\sin(\theta_t - \alpha_t), \cos(\theta_t - \alpha_t)). \tag{63}$$

The continuous error is then mapped to the nearest discrete grid point. The corresponding policy entry at time $t \bmod T$ returns a discrete control index, which is converted back to the associated linear and angular velocity values from the control grids.

Algorithm 5 GPI Lookup

function GPICONTROLLER(x_t, x_t^{ref}, t)
  Compute tracking error $e_t = x_t - x_t^{\text{ref}}$
  Wrap heading error to $[-\pi, \pi)$
  Map e_t to nearest discrete error-state index
  Retrieve policy action at time index $t \bmod T$
  Convert action index to (v_t, ω_t)
  return control input (v_t, ω_t)
end function

GPI separates expensive stochastic planning from online control. This can be especially advantageous if a robot was

deployed into a known environment with low memory or compute. Then, a policy can be computed offline and a lookup table is implemented for online pathfinding.

IV. Results

We provide the results for the CEC algorithm. The red dotted circles in the image represent the true collision boundary after accounting for the robot size.

A. Deterministic CEC

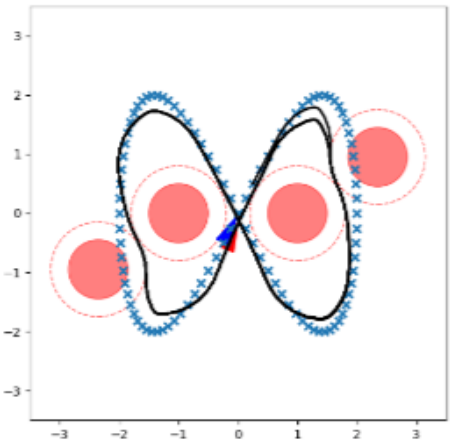


Fig. 1: CEC trajectory without motion noise.

Figure 1 shows that the CEC planner finds a collision-free trajectory when the motion noise is removed. Since the deterministic simulation matches the model used inside the optimizer, the controller can closely follow the reference trajectory without fear of colliding into the obstacles. The planner was also efficient as it did not need a large time-horizon to achieve accurate tracking.

B. CEC with Noise and No Additional Margin

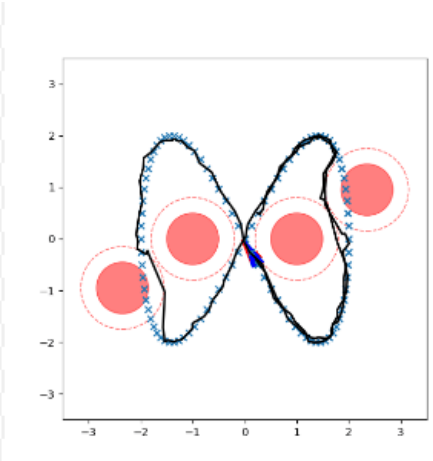


Fig. 2: CEC trajectory with motion noise and no additional safety margin.

Figure 2 shows that using no additional noise margin causes the robot to follow a path similar to the deterministic case. However, this trajectory leads to several scrape collisions because the optimizer does not explicitly account for stochastic motion noise. Although the trajectory is feasible, it would likely cause damage to the exterior of the robot.

C. CEC with Large Noise Margin

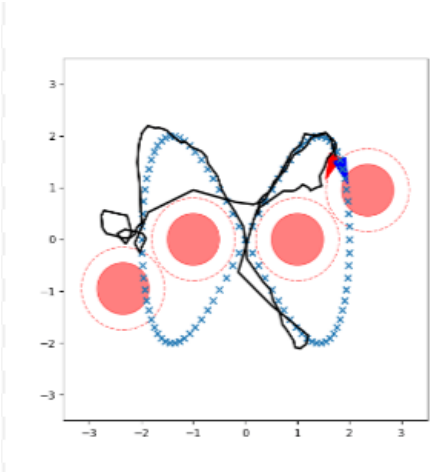


Fig. 3: CEC trajectory with motion noise and a 0.4m additional safety margin.

Figure 3 shows the effect of increasing the obstacle margin by 0.4m. This larger inflation closes the tight gap between the central obstacles, forcing the optimizer to choose a safer route through the center of the environment.

D. CEC with Moderate Noise Margin

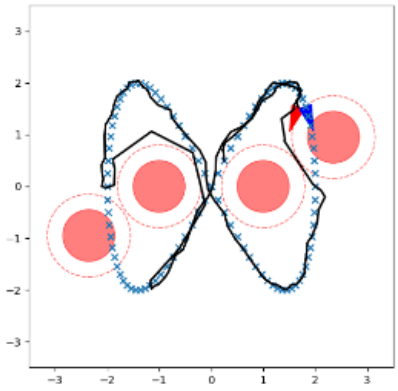


Fig. 4: CEC trajectory with motion noise and moderate additional safety margin.

Figure 4 provides a supporting comparison to Fig. 3. With a smaller safety margin, the planner still becomes more conservative than the no-margin case, but it does not close the

tight obstacle gaps as strongly as the 0.4m margin. This can be seen by how, on the right side of the trajectory, the robot still passes between two inflated obstacle regions. However, this movement was in one motion, which avoids any collision.

E. GPI without Motion Noise

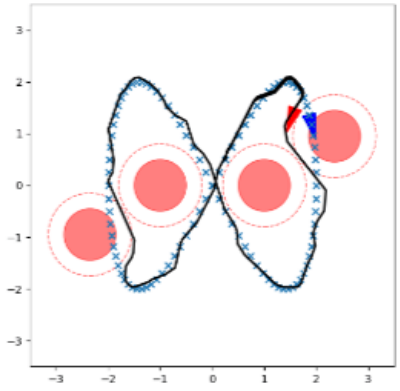


Fig. 5: GPI trajectory without motion noise.

Figure 5 shows the trajectory generated by the GPI controller in the absence of motion noise. Similar to the deterministic CEC result, the policy successfully tracks the reference trajectory while avoiding obstacles.

Unlike the smooth trajectories generated by the CEC controller, the GPI trajectory exhibits noticeable discretization effects. This produces abrupt changes in steering angle and velocity, causing the robot orientation and position to appear significantly more jumpy than the CEC controller.

F. GPI with Motion Noise no Margin

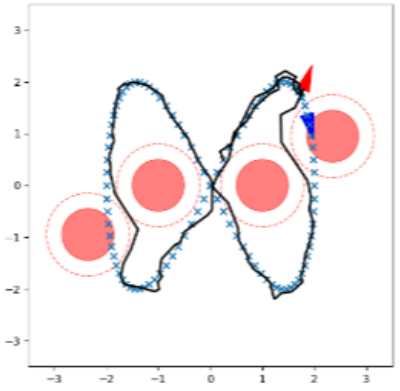


Fig. 6: GPI trajectory with motion noise and no margin.

Figure 6 shows the behavior of the GPI controller under stochastic motion noise. Because there is no margin. the robot still continue to take the riskier path between the two circles.

G. GPI with Motion Noise and Margin

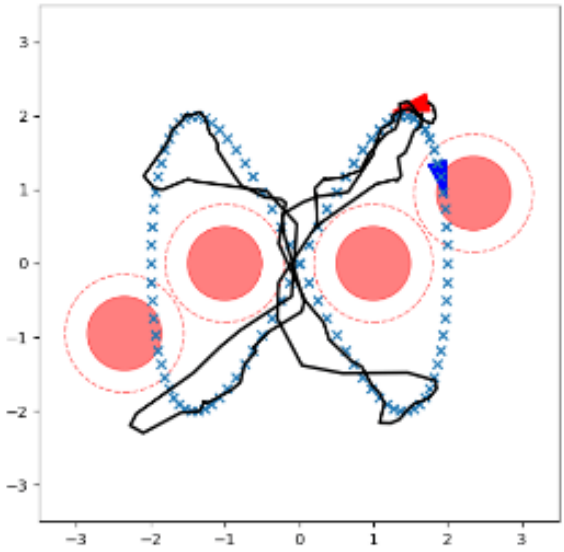


Fig. 7: GPI trajectory with increased collision margin.

Through increasing the collision margin, we see that the orbot's behavior changes, preferring to pass through the center rather than take the riskier route between the circles.

Through the visuals, we have that a collision margin of approximately 0.15m provided the best balance between safety and tracking performance. Smaller margins led to more collisions under noise, while larger margins created erratic behavior that led to significantly increased error.

H. Effect of Policy Evaluation Iterations

An additional tuning parameter in the GPI algorithm is the number of policy-evaluation sweeps performed before each policy-improvement step. Increasing the number of evaluation sweeps initially improved policy quality because the value function more accurately reflected the long-term consequences of each action.

However, beyond a certain point, additional policy-evaluation sweeps instead led to degraded performance. This behavior is likely caused by approximation errors introduced by state discretization. As the value function converges, discretization errors become more pronounced and can propagate throughout the state space, leading to worse tracking performance. Therefore, an intermediate number of policy evaluations produced the best results.

I. Discussion

The CEC and GPI results show that safe trajectory tracking is relatively straightforward in the deterministic setting but becomes much more difficult under stochastic motion noise.

Without noise, both controllers can find collision-free trajectories because the predicted motion model matches the executed trajectory. In this case, the robot can pass close to the obstacle boundaries and track the reference closely.

When motion noise is introduced, the same trajectory is no longer reliable. This noise can push the robot into nearby obstacles, especially between the narrow gaps between circular obstacles. As a result, the controller must treat nearby obstacles more conservatively. In cases like these, circles that may appear as two separate obstacles with a tight gap should be treated as one larger obstacle. This makes the center path through the environment the most viable route for safely following the reference trajectory with noise.

The safety margin was one of the most important tuning parameters for both CEC and GPI. Margins that were too small allowed the robot to pass between the obstacles, which led to collisions with noise. Margins that were too large forced the robot to deviate too much from the reference trajectory.

Another result was that that the best horizon or evaluation depth was not always the largest one. For CEC, increasing the planning horizon gives the optimizer more future information, but it can also lead to decreased performance and erratic behavior. For GPI, increasing the number of policy-evaluation sweeps initially improved the value estimate, but too many sweeps could amplify discretization errors. In both implementations, limiting the planning scopes generally produced better results than the largest horizon.

Overall, getting the right results required significant fine-tuning. CEC needed obstacle inflation and warm-starting to produce reliable trajectories, while GPI required careful discretization, collision penalties, and policy-evaluation tuning.